

Figure 7: Token registration

```
mountpoints_container      "dc=MountPoints,
dc=example,dc=com"
domains_container         "dc=Domains, dc=example,dc=com"
clients_container         "dc=Clients,
dc=example,dc=com"
```

Now we are ready to start the “webadm” service. If you want to change the port it listens on, you can modify `/opt/webadm/bin/webadm`. By default, the Web server listens on port 443 (SSL), and the SOAP server on ports 8080 and 8443 (SSL). The settings are `HTTP_PORT`, `SOAP_PORT`, `SSL` etc. To start the service, run:

```
#/opt/webadm/bin/webadm start
```

Navigate to WebADM (`https://<your-server-address>`). Only the user(s) given as `super_admins` in `webadm.conf` can run the graphical set-up. So, log in with your `admin` account and run the graphical set-up via the *Setup* button that appears in the home page.

WebADM requires a DN-based login until the set-up is completed. Then it will use the login mode as configured in `webadm.conf`. The graphical set-up process will:

- Create the required database tables (as specified in `conf/database.xml`).
- Register the required LDAP schema *objectclasses* and attributes.
- Create the proxy user (if it does not already exist).
- Create the WebADM LDAP containers (as defined in `conf/webadm.conf`).

This article will not describe in detail how to use the WebADM GUI—for that, take a look at the WebADM manual. This GUI is the centralised point for LDAP and OpenOTP administration activities like creation, deletion, modification, etc.

Note: In my test set-up, I’m using a single machine as the OpenOTP, OpenLDAP and RADIUS Bridge server, but for a production environment, it’s suggested that you use separate servers for each service.

Once you log in to the console, you can see that some domains have already been created, like `dc=clients,dc=MountPoints,dc=ObjectSets,dc=WebApps,dc=WebSrvs`, etc. These are created by the set-up script for use by WebADM.

Test user

Now we have to create a user for our test set-up. For this, let’s create a different OU called *People*. Once the OU is created, you can see it in the left pane. To create the WebADM user, follow the steps shown below:

1. Click *Create*, and you will see the screen shown in Figure 3.
2. Select *WebADM Account* and click *Proceed*. Provide all necessary information, like *CN*, *Last Name*, *Login Name*, *password*, etc. Then click *Proceed* and click *Create the object*.
3. After creating *testuser*, Figure 4 is the screen you will see.
4. Now we change *OpenOTP* settings like *Token*, *PIN*, etc. Click the *Application* menu, select *OTP Authentication Server* and click *Configure*. You will see the screen shown in Figure 5.
5. Now configure the OpenOTP application as per your needs, like the default OpenOTP authentication mode, domain name, host-name of your server, etc. The basic setting is *Login Mode* (authentication mode). I used *LDAPOTP*, to require both LDAP and OTP passwords, which is the default. If you enable *Password Swapping*, it will accept either LDAP or OTP.
6. Now click *Proceed*, then click *testuser* and click *OTP Server*. You will see the screen shown in Figure 6.
7. Select *Register/Unregister Token*; you will see the screen shown in Figure 7.
8. Select *Mobile OTP(Time-Based)*. If you have an existing key, you can put it in the *Token Key* field, or you can let the server generate a key.
9. Then you enter your *Token PIN*, click *Register*, and you are done.

So now our test user is ready to use Mobile-OTP. You can download the relevant application from your Apps store. (I’m using ‘Droid OTP’ for my Android device.) Launch it, and enter your secret PIN to generate the passwords you can use to log in to different applications. Next, let us look into how to configure applications like SSH and RADIUS.

Configuring SSHD for OTP passwords

Download *libopenotp-1.0.4-1.tgz* and *pam_openotp-1.0.4.tar.gz* from `http://www.rcdevs.com/downloads/` and decompress them. For 32-bit architectures, the default modules are present in the *build_linux32* sub-folder of the extracted *libopenotp* tar, but for 64-bit, you have